

date information to respond to the query.

5. Augmentation of Context:

By merging two knowledge streams - the fixed, general knowledge embedded in the LLM and the flexible, domain-specific information augmented on demand as an additional layer of context, aligns the Large Language Model (LLM) with both established and emerging information.

6. Generation of Response by LLM:

The *context-infused prompt*, consisting of the original user query combined with the retrieved relevant content is provided to a Large Language Model (LLM) like GPT, T5 or Llama. The LLM then processes this augmented in-

date information to respond to the query.

5. Augmentation of Context:

By merging two knowledge streams - the fixed, general knowledge embedded in the LLM and the flexible, domain-specific information augmented on demand as an additional layer of context, aligns the Large Language Model (LLM) with both established and emerging information.

6. Generation of Response by LLM:

The *context-infused prompt*, consisting of the original user query combined with the retrieved relevant content is provided to a Large Language Model (LLM) like GPT, T5 or Llama. The LLM then processes this augmented in-



effective when dealing with stable data or when the model needs to adhere to a specific tone and style.

Drawbacks: Fine-tuning is computationally expensive and often requires substantial resources for initial training. Additionally, it risks overfitting if the dataset is too narrow, making the model less generalizable.

Use Case Examples:

- **Medical Diagnosis:** A fine-tuned model on medical datasets becomes highly specialized in understanding and generating medical advice based on specific terminologies and contexts.
- **Customer Support:** For a software company, fine-tuning on company-specific troubleshooting protocols ensures high-accuracy and consistent responses tailored to user queries.

2.2.2 RAG: Dynamic Information and Large Knowledge Bases Retrieval-Augmented Generation (RAG) combines LLMs with a retrieval mechanism that allows the model to access external data sources in real-time, making it suitable for scenarios requiring up-to-date or frequently changing information. RAG systems are valuable for handling vast knowledge bases where embedding all the

1. ACCURATE TEXT EXTRACTION:

Since PDFs can have complex formatting, it is essential to use reliable tools and methods to convert the PDF content into usable text for further processing.

- **Appropriate Tool for Extraction:** There are tools and libraries for extracting text from PDFs for most popular programming languages (i.e: `pdfplumber` or `PyMuPDF (fitz)` for Python). These libraries handle most common PDF structures and formats, preserving the text's layout and structure as much as possible.
- **Verify and Clean Extracted Text:** After extracting text, always verify it for completeness and correctness. This step is essential for catching any extraction errors or artifacts from formatting.

Use Case Examples:

- **Financial Advisor Chatbot:** Using RAG, a chatbot can pull the latest market trends and customer-specific portfolio data to offer personalized investment advice.
- **Legal Document Analysis:** RAG can retrieve relevant case laws and statutes from a constantly updated database, making it suitable for legal applications where accuracy and up-to-date information are critical.

2.2.3 When to Use Base Models Using base models (without fine-tuning or RAG) is appropriate when the task requires broad generalization, low-cost deployment, or rapid prototyping. Base models can handle simple use cases like generic customer support or basic question answering, where specialized or dynamic information is not required.

5/36

Use Case Examples:

- **Financial Advisor Chatbot:** Using RAG, a chatbot can pull the latest market trends and customer-specific portfolio data to offer personalized investment advice.
- **Legal Document Analysis:** RAG can retrieve relevant case laws and statutes from a constantly updated database, making it suitable for legal applications where accuracy and up-to-date information are critical.

2.2.3 When to Use Base Models Using base models (without fine-tuning or RAG) is appropriate when the task requires broad generalization, low-cost deployment, or rapid prototyping. Base models can handle simple use cases like generic customer support or basic question answering, where specialized or dynamic information is not required.



effective when dealing with stable data or when the model needs to adhere to a specific tone and style.

Drawbacks: Fine-tuning is computationally expensive and often requires substantial resources for initial training. Additionally, it risks overfitting if the dataset is too narrow, making the model less generalizable.

Use Case Examples:

- **Medical Diagnosis:** A fine-tuned model on medical datasets becomes highly specialized in understanding and generating medical advice based on specific terminologies and contexts.
- **Customer Support:** For a software company, fine-tuning on company-specific troubleshooting protocols ensures high-accuracy and consistent responses tailored to user queries.

2.2.2 RAG: Dynamic Information and Large Knowledge Bases Retrieval-Augmented Generation (RAG) combines LLMs with a retrieval mechanism that allows the model to access external data sources in real-time, making it suitable for scenarios requiring up-to-date or frequently changing information. RAG systems are valuable for handling vast knowledge bases where embedding all the

. Preprocessing and Cleaning:

Preprocessing the extracted text is key for removing noise that could affect the performance of both retrieval and generative models. Proper cleaning ensures the text is consistent, relevant, and ready for further processing.

- **Remove Irrelevant Content:** Use regular expressions or NLP-based rules to clean up non-relevant content like headers, footers, page numbers, and any repeating text that doesn't contribute to the document's meaning.
- **Normalize Text:** Standardize the text format by converting it to lower-case, removing special characters, and trimming excessive whitespace. This normalization helps create consistent input for the retrieval models.

. Utilizing PDF Metadata and Annotations:

PDFs often contain metadata (such as the author, title, and creation date) and

paragraphs as the primary chunking unit.



3. Preprocessing and Cleaning:

Preprocessing the extracted text is key for removing noise that could affect the performance of both retrieval and generative models. Proper cleaning ensures the text is consistent, relevant, and ready for further processing.

- **Remove Irrelevant Content:** Use regular expressions or NLP-based rules to clean up non-relevant content like headers, footers, page numbers, and any repeating text that doesn't contribute to the document's meaning.
- **Normalize Text:** Standardize the text format by converting it to lower-case, removing special characters, and trimming excessive whitespace. This normalization helps create consistent input for the retrieval models.

4. Utilizing PDF Metadata and Annotations:

PDFs often contain metadata (such as the author, title, and creation date) and

any coherent pieces can improve retrieval performance.

- **Semantic Chunking:** Instead of splitting text arbitrarily, use semantic chunking based on logical divisions within the text, such as paragraphs or sections. This ensures that each chunk retains its context, which is important for both retrieval accuracy and relevance.
- **Dynamic Chunk Sizing:** Adjust the chunk size according to the content type and the model's input limitations. For example, scientific documents might be chunked by sections, while other types of documents could use paragraphs as the primary chunking unit.

3. *Preprocessing and Cleaning:*

Preprocessing the extracted text is key for removing noise that could affect the performance of both retrieval and generative models. Proper cleaning ensures the text is consistent, relevant, and ready for further processing.

any coherent pieces can improve retrieval performance.

- **Semantic Chunking:** Instead of splitting text arbitrarily, use semantic chunking based on logical divisions within the text, such as paragraphs or sections. This ensures that each chunk retains its context, which is important for both retrieval accuracy and relevance.
- **Dynamic Chunk Sizing:** Adjust the chunk size according to the content type and the model's input limitations. For example, scientific documents might be chunked by sections, while other types of documents could use paragraphs as the primary chunking unit.

3. Preprocessing and Cleaning:

Preprocessing the extracted text is key for removing noise that could affect the performance of both retrieval and generative models. Proper cleaning ensures the text is consistent, relevant, and ready for further processing.

case, removing special characters, and trimming excessive whitespace. This normalization helps create consistent input for the retrieval models.

4. Utilizing PDF Metadata and Annotations:

PDFs often contain metadata (such as the author, title, and creation date) and annotations that provide additional context, which can be valuable for retrieval tasks in RAG applications.

- **Extract Metadata:** You can use tools specific to programming languages like PyMuPDF or `pdfminer.six` for Python to extract embedded metadata. This metadata can be used as features in retrieval models, adding an extra layer of context for more precise search results.
- **Utilize Annotations:** Extract and analyze annotations or comments within PDFs to understand important or highlighted sections. This can help prioritize content in the retrieval process.

5. Error Handling and Reliability:

Reliability in processing PDFs is essential for maintaining the stability and reliability of RAG applications. Implementing proper error handling and logging

- **Utilize Annotations.** Extract and analyze annotations or comments within PDFs to understand important or highlighted sections. This can help prioritize content in the retrieval process.

5. Error Handling and Reliability:

Reliability in processing PDFs is essential for maintaining the stability and reliability of RAG applications. Implementing proper error handling and logging helps manage unexpected issues and ensures smooth operation.

- **Implement Error Handling:** Use try-except blocks to manage potential errors during PDF processing. This ensures the application continues running smoothly and logs any issues for later analysis.

Table 2: Comparison of RAG Approaches: OpenAI vs. Llama

Feature	OpenAI's Assistant API (GPT Series)	Llama (Open-Source LLM Model)
Ease of Use	High. Simple API calls with no model management	Moderate. Requires setup and model management
Customization	Limited to prompt engineering and few-shot learning	High. Full access to model fine-tuning and adaptation
Cost	Pay-per-use pricing model	Upfront infrastructure costs; no API fees
Deployment Flexibility	Cloud-based; depends on OpenAI's infrastructure	Highly flexible; can be deployed locally or in any cloud environment
Performance	Excellent for a wide range of general NLP tasks	Excellent, particularly when fine-tuned for specific domains
Security and Data Privacy	Data is processed on OpenAI servers; privacy concerns may arise	Full control over data and model; suitable for sensitive applications
Support and Maintenance	Strong support, documentation, and updates from OpenAI	Community-driven; updates and support depend on community efforts
Scalability	Scalable through OpenAI's cloud infrastructure	Scalable depending on infrastructure setup
Control Over Up-	Limited; depends on Ope-	Full control; users can decide

THE USE OF EMBEDDING TRANSFORMERS, which capture the semantic meaning of the text. The embeddings are stored in a vector database optimized for fast similarity-based retrieval.

The RAG system architecture consists of two key components: a retriever, which converts user queries into vector embeddings to search the vector database, and a generator, which synthesizes the retrieved content into a coherent, factual response. Two types of models are considered: OpenAI's GPT models, accessed through the Assistant API for ease of integration, and the open-source Llama model, which offers greater customization for domain-specific tasks.

then segmented into manageable chunks, ensuring efficient retrieval. These text segments are converted into vector embeddings using transformer-based models like BERT or Sentence Transformers, which capture the semantic meaning of the text. The embeddings are stored in a vector database optimized for fast similarity-based retrieval.

The RAG system architecture consists of two key components: a retriever, which converts user queries into vector embeddings to search the vector database, and a generator, which synthesizes the retrieved content into a coherent, factual response. Two types of models are considered: OpenAI's GPT models, accessed through the Assistant API for ease of integration, and the open-source Llama model, which offers greater customization for domain-specific tasks.

In developing the system, several challenges are addressed, such as managing complex PDF layouts (e.g., multi-column formats, embedded images) and maintaining retrieval efficiency as the knowledge base grows. These challenges were highlighted during a preliminary evaluation process, where participants pointed out the difficulty of handling documents with irregular structures. Feedback from

- **Utilize Annotations.** Extract and analyze annotations or comments within PDFs to understand important or highlighted sections. This can help prioritize content in the retrieval process.

5. Error Handling and Reliability:

Reliability in processing PDFs is essential for maintaining the stability and reliability of RAG applications. Implementing proper error handling and logging helps manage unexpected issues and ensures smooth operation.

Implement Error Handling: Use try-except blocks to manage potential errors during PDF processing. This ensures the application continues running smoothly and logs any issues for later analysis.

multi-modal operations (such as text, images, audio, and video inputs) by combining text generation with file searches, code execution, and API calls. For a RAG system, this means an assistant can retrieve documents, generate vector embeddings, search for relevant content, augment user queries with additional context, and generate responses—all in a seamless, integrated workflow. It includes memory management across sessions, so the assistant *remembers* past queries, retrieved documents, or instructions. Assistants can be configured with specialized instructions, behaviors, parameters other than custom tools that makes this API far more powerful for developing RAG systems.

This subsection provides a step-by-step guide and code snippets to utilize the OpenAI's **File Search tool** within the Assistant API, as illustrated in

multi-modal operations (such as text, images, audio, and video inputs) by combining text generation with file searches, code execution, and API calls. For a RAG system, this means an assistant can retrieve documents, generate vector embeddings, search for relevant content, augment user queries with additional context, and generate responses—all in a seamless, integrated workflow. It includes memory management across sessions, so the assistant *remembers* past queries, retrieved documents, or instructions. Assistants can be configured with specialized instructions, behaviors, parameters other than custom tools that makes this API far more powerful for developing RAG systems.

This subsection provides a step-by-step guide and code snippets to utilize the OpenAI's **File Search tool** within the Assistant API, as illustrated in

Best Practice: Ask the Model to adopt a Persona Provide specialized context-Rich *instructions* that guide the assistant on how to handle queries, what tone to use (e.g., formal, friendly), and which domains to prioritize. This ensures the assistant generates more accurate and contextually appropriate responses.

Best Practice: Use inner monologue or Conversation Structure The idea of inner monologue as a part of *instructions* is to instruct the model to put parts of the output into a structured format. This help understand the reasoning process that a model uses to arrive at a final answer.

Best Practice: Fine-Tune Model Parameters Based on Use Case Adjust parameters such as *temperature* (controls randomness) and *top p* (controls diversity) based on the application needs. This can impact the coherence and creativity of the assistant's outputs. For a customer support assistant, a lower temperature may be preferable for consistent responses, while a more creative application might benefit from a higher temperature.

Best Practice: Classifying Queries into Categories For tasks in which lots of independent sets of instructions are needed to handle different cases, it can be beneficial to first classify the type of query and to use that classification to determine which instructions are needed.

Mistake: Ignoring context and query augmentation strategies Relying solely on the vector embeddings without considering query-specific context or augmentation can lead to suboptimal responses.

Best Practice: Augment Queries with Contextual Information Incorporate additional contextual information when forming queries to improve retrieval quality. Using techniques like relevance feedback or pseudo-relevance feedback can help refine search results. [[6]] [[1]]

Best Practice: Handle Naming Conflicts Gracefully When creating vector stores, consider adding a timestamp or unique identifier to the vector store name to avoid naming conflicts and make it easier to manage multiple vector stores.

Best Practice: Chunking strategy By default OpenAI uses a *max chunk size tokens of 800* and *chunk overlap tokens of 400* to chunk the file(s) for Vector Stores. Properly sized chunks ensure that each chunk contains a coherent and contextually meaningful piece of information. If chunks are too large, they may contain unrelated content, conversely, if chunks are too small, they may lack sufficient context to be useful. Configure the variables accordingly the PDF(s).

optimally with its specific requirements.

4.2 Two Approaches to RAG: Proprietary and Open source

This section introduces a structured guide for developing Retrieval Augmented Generation (RAG) systems, focusing on two distinct approaches: *using OpenAI's Assistant API (GPT Series)* and an *open-source Large Language Model (LLM) Llama* and thus divided into two subsections [4.2.1][4.2.2]. The objective is to equip developers with the knowledge and practical steps necessary to implement RAG systems effectively, while highlighting common mistakes and best practices at each stage of the process. Each subsection is designed to provide practical insights into setup, development, integration, customization and optimization to generate well-grounded and aligned outputs.

In addition to the two primary approaches discussed in this guide there are several alternative frameworks and methodologies for developing Retrieval Augmented Generation (RAG) systems. Each of these options such as Cohere, AI21's Jurassic-2, Google's PaLM, and Meta's OPT have their merits and trade-offs in terms of deployment flexibility, cost, ease of use, and performance.

optimally with its specific requirements.

4.2 Two Approaches to RAG: Proprietary and Open source

This section introduces a structured guide for developing Retrieval Augmented Generation (RAG) systems, focusing on two distinct approaches: *using OpenAI's Assistant API (GPT Series)* and an *open-source Large Language Model (LLM) Llama* and thus divided into two subsections[4.2.1][4.2.2]. The objective is to equip developers with the knowledge and practical steps necessary to implement RAG systems effectively, while highlighting common mistakes and best practices at each stage of the process. Each subsection is designed to provide practical insights into setup, development, integration, customization and optimization to generate well-grounded and aligned outputs.

In addition to the two primary approaches discussed in this guide there are several alternative frameworks and methodologies for developing Retrieval Augmented Generation (RAG) systems. Each of these options such as Cohere, AI21's Jurassic-2, Google's PaLM, and Meta's OPT have their merits and trade-offs in terms of deployment flexibility, cost, ease of use, and performance.

will enable seamless question generation without relying on external API services, ensuring both privacy and computational efficiency.

1. Install the following Python libraries

```
pip install pymupdf langchain-huggingface faiss-cpu  
pip install oLlama sentence-transformers sentencepiece  
pip install langchain-community
```

2. Converting PDFs to Text Files

Save this script as `pdf_to_text.py`. This script converts PDF files in a given folder into text files. You have to create one folder at the same directory. The folder name should be *Data*. You have to keep your PDF files in this *Data* folder for the further process. Check the GitHub Link of the Code.

Code Example:

```
import os
```


Code Example:

```
import os
import fitz # PyMuPDF for reading PDFs

def convert_pdfs_to_text(pdf_folder, text_folder):
    if not os.path.exists(text_folder):
        os.makedirs(text_folder)

    for file_name in os.listdir(pdf_folder):
        if file_name.endswith(".pdf"):
            file_path = os.path.join(pdf_folder, file_name
            )
```

Common Mistakes and Best Practices

Incompatible embeddings The FAISS index is typically created using a specific embeddings model. If a different embeddings model is used during querying (e.g., a different version of `sentence-transformers`), it may lead to retrieval mismatches or errors. Always ensure that the same model is used both during indexing and querying.

Model version issues Using an incorrect or unsupported model version (e.g., referencing a non-existent version like `Llama 4.5`) can lead to failures during the model loading process. Always verify that the model version you are using is supported and available.

Overly general prompts Prompts that are too broad or generic can result in vague or irrelevant responses from the model. Craft precise and targeted prompts to ensure more accurate and relevant answers.

Ignoring context Language models can generate incorrect or hallucinated responses when the retrieved documents lack the necessary information, leading the model to fill in gaps inaccurately. Always ensure sufficient context is provided in the query.

Memory leaks Extended use of FAISS and Ollama in continuous loops without proper memory management can result in memory leaks, gradually consuming system resources. Monitor memory usage and free up resources after each loop to avoid system slowdowns.

Model re-initialization Reloading or re-initializing models unnecessarily can slow down your system. Reuse initialized models whenever possible to improve system efficiency and reduce overhead.

```

        store with the model
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=vector_store.as_retriever(),
    chain_type_kwargs={"prompt": prompt}
)

return qa_chain

# Function to run the RAG system with a user question
def get_answer(question, qa_chain):
    answer = qa_chain.run(question)
    return answer

if __name__ == "__main__":
    # Path to the FAISS index directory
    index_path = "DataIndex"

    # Initialize the RAG system
    rag_system = create_rag_system(index_path)

    # Get user input and generate the answer

```



```
        clearly state that the information is not
        available in the context provided.
    If possible, provide a step-by-step explanation and
    highlight key details.
    """

    # Create a template for formatting the input for the
    model
    prompt = PromptTemplate(
        input_variables=["context", "question"],
        template=prompt_template
    )

    # Create a RetrievalQA chain that combines the vector
    store with the model
    qa_chain = RetrievalQA.from_chain_type(
        llm=llm,
        chain_type="stuff",
        retriever=vector_store.as_retriever(),
        chain_type_kwargs={"prompt": prompt}
    )

    return qa_chain
```

```

        text_file_name = os.path.splitext(file_name)
            [0] + ".txt"
        text_file_path = os.path.join(text_folder,
            text_file_name)

        with fitz.open(file_path) as doc:
            text = ""
            for page in doc:
                text += page.get_text()

        with open(text_file_path, "w", encoding="utf-8") as text_file:
            text_file.write(text)
        print(f"Converted {file_name} to {
            text_file_name}")

if __name__ == "__main__":
    pdf_folder = "Data"
    text_folder = "DataTxt"
    convert_pdfs_to_text(pdf_folder, text_folder)

```

Functionality: Converts all PDFs in the Data folder to text files and saves



that, we implemented error handling to throw meaningful errors to the user in the code snippets provided to seamlessly run the code.

2	anonymous	Well structured of instructions but copy code should be with terminal command
3	anonymous	Copy-pasting code from the pdf files.
4	anonymous	Some code problems...
5	anonymous	it is very easy to understand
6	anonymous	Havent face any challenges
7	anonymous	Editor and environment issues not relating to the guide.

Fig. 7: Feedback on challenges faced during the implementation of the guide

graphs. This integration aims to improve the factual accuracy and reasoning capabilities of the models, making them more reliable for knowledge-intensive tasks[8].

4. **Adaptive Learning and Continual Fine-Tuning:** There is a growing interest in adaptive learning techniques that allow RAG models to continuously fine-tune themselves based on new data and user feedback. This approach aims to keep models up-to-date and relevant in rapidly changing information environments[7].
5. **Cross-Lingual and Multimodal Capabilities:** Future RAG models are expected to expand their capabilities across different languages and modalities. Incorporating cross-lingual retrieval and multimodal data processing can make RAG models more versatile and applicable to a wider range of global and multimedia tasks[2].

Future research will likely focus on enhancing their adaptability, cross-lingual capabilities, and integration with diverse data sources to address increasingly complex information needs.

7 Conclusions

of AI and NLP technologies. As these models and tools improve, there are many potential areas for growth, such as finding better ways to search for information, adapting to new data automatically, and handling more than just text (like images or audio). Recent advancements in tools and technologies have further accelerated the development and deployment of RAG models. As RAG models continue to evolve, several emerging trends are shaping the future of this field.

1. **Haystack:** An open-source framework that integrates dense and sparse retrieval methods with large-scale language models. Haystack supports real-time search applications and can be used to develop RAG models that perform tasks such as document retrieval, question answering, and summarization [4].
2. **Elasticsearch with Vector Search:** Enhanced support for dense vector search capabilities, allowing RAG models to perform more sophisticated retrieval tasks. Elasticsearch's integration with frameworks like Faiss enables

12. Any other comments or suggestions for future improvement?

3 Responses

ID ↑	Name	Responses
1	anonymous	My last remark is about the vectoreStore, I just found out it is stored in OpenAI's server, so there should be a warning to not upload any sensitive documents. And also, I want to know , in the guide: where it is stored, and how to delete it.

Fig. 8: Comments and suggestions for improving the guide

trustworthiness in specific scenarios, we were able to collect meaningful feedback and directly address areas of difficulty faced by practitioners. The feedback-driven improvements have not only made the guide more user friendly but also demonstrated the importance of continuous iteration based on real world use

33/36

the discussions with peers and instructors as a key takeaway.

5.4 Incorporating Feedback to Improve the Guide

The evaluation also revealed opportunities for improvement of guide, particularly in enhancing the clarity of instructions and streamlining the implementation process. The most common issues raised were technical, most of them related to copying from PDF file generating errors as number of lines. In addition to that, we implemented error handling to throw meaningful errors to the user in the code snippets provided to seamlessly run the code.

2	anonymous	Well structured of instructions but copy code should be with terminal command
3	anonymous	Copy-pasting code from the pdf files.
4	anonymous	Some code problems...
5	anonymous	it is very easy to understand
6	anonymous	Havent face any challenges
7	anonymous	Editor and environment issues not relating to the guide.

graphs. This integration aims to improve the factual accuracy and reasoning capabilities of the models, making them more reliable for knowledge-intensive tasks[8].

4. **Adaptive Learning and Continual Fine-Tuning:** There is a growing interest in adaptive learning techniques that allow RAG models to continuously fine-tune themselves based on new data and user feedback. This approach aims to keep models up-to-date and relevant in rapidly changing information environments[7].
5. **Cross-Lingual and Multimodal Capabilities:** Future RAG models are expected to expand their capabilities across different languages and modalities. Incorporating cross-lingual retrieval and multimodal data processing can make RAG models more versatile and applicable to a wider range of global and multimedia tasks[2].

Future research will likely focus on enhancing their adaptability, cross-lingual capabilities, and integration with diverse data sources to address increasingly complex information needs.

7 Conclusions